

From Iterative Optimization to Real-Time Feed-Forward Style Transfer:

A Direct Comparison of Gatys et al. (2016) and Adaptive Instance Normalization (2017)

Abstract

We compare Gatys et al.’s (2016) *Image Style Transfer Using Convolutional Neural Networks* and Huang and Belongie’s (2017) *Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization*, evaluating the effectiveness of each method both mathematically and in practice.

1 Introduction

Neural style transfer was first formulated by Gatys, Ecker, and Bethge (2016) as a slow, image-specific optimization problem driven by a Gram-matrix style loss and a feature-reconstruction content loss. Huang and Belongie’s Adaptive Instance Normalization (AdaIN, 2017) reformulated the same underlying goal as a single feed-forward pass through a trained encoder-decoder network, replacing the Gram matrix with a much cheaper channel-wise statistic-alignment operation. This report gives a direct, mechanism-level and mathematical comparison of the two methods, focused on three axes: (1) the usage of the secondary statistic Gram matrix in Gatys as opposed to both secondary(sigma) and primary(mean) in AdaIn (2) AdaIn architecture is per-channel whereas Gatys is cross-channel. (3) the architectural shift from per-image iterative optimization to amortized, real-time feed-forward inference. We show that these three differences are not independent design choices but a single causal chain: the change in style representation is precisely what makes the change in computational paradigm possible.

2 Gatys et al. (2016): Gram-Matrix Style Loss and Iterative Optimization

2.1 Style representation: the Gram matrix

For a feature map at layer l with C_l channels and $H_l \times W_l$ spatial positions, Gatys et al. reshape the activations into a matrix $F^l \in \mathbb{R}^{C_l \times N_l}$, where $N_l = H_l W_l$. The style representation at that layer is the Gram matrix

$$G_{ij}^l = \sum_{k=1}^{N_l} F_{ik}^l F_{jk}^l, \quad (1)$$

a $C_l \times C_l$ matrix whose entry G_{ij}^l measures the correlation between the activations of channel i and channel j , pooled over all spatial locations. This discards spatial layout entirely (the sum runs over all positions k) and retains only *which features co-occur, and how strongly*, independent of where they occur – which is exactly the property needed to represent texture/style rather than layout. Crucially, as we can interpret, we could have also used only the mean or variance or any

other activation statistic that disregards the spatial arrangement of the individual activations instead of the gram matrix.

The style loss aggregates squared error between Gram matrices of the generated image p and the style image s , summed across multiple layers $l \in \mathcal{L}$ (typically relu1_1 through relu5_1):

$$\mathcal{L}_{\text{style}}(p, s) = \sum_{l \in \mathcal{L}} w_l \frac{1}{4C_l^2 N_l^2} \sum_{i,j} \left(G_{ij}^l(p) - G_{ij}^l(s) \right)^2. \quad (2)$$

Content loss, in contrast, is computed at a single deeper layer as ordinary squared error between raw feature activations:

$$\mathcal{L}_{\text{content}}(p, c) = \frac{1}{2} \sum_{i,k} \left(F_{ik}^l(p) - F_{ik}^l(c) \right)^2, \quad l = \text{relu4_2 (typical choice)}. \quad (3)$$

2.2 Optimization procedure

Crucially, Gatys et al. do not train the network. VGG-19 is used purely as a fixed, differentiable measuring instrument. The *pixels of the output image itself* are the optimized variable:

$$p^* = \arg \min_p \mathcal{L}_{\text{content}}(p, c) + \alpha \mathcal{L}_{\text{style}}(p, s), \quad (4)$$

typically solved by initializing p from white noise (or from c) and running L-BFGS or Adam for several hundred iterations, backpropagating gradients through the frozen VGG all the way to the pixels. Every new (content, style) pair requires restarting this optimization from scratch: there is no learned, reusable mapping – only a per-image search procedure. This is why the method, despite producing high-quality results, is computationally expensive: typically on the order of minutes per image on a GPU, since each of several hundred iterations requires a full forward and backward pass through VGG.

3 AdaIN (2017): Channel-Wise Statistic Alignment and Feed-Forward Inference

3.1 Style representation: adaptive instance normalization

Huang and Belongie replace the $C_l \times C_l$ Gram matrix with a far lighter-weight statistic: the per-channel mean $\mu(\cdot) \in \mathbb{R}^{C_l}$ and standard deviation $\sigma(\cdot) \in \mathbb{R}^{C_l}$ of a single feature map, computed over spatial positions for each channel independently. Given content features $f(c)$ and style features $f(s)$ at a single fixed layer (relu4_1), the AdaIN operation is

$$\text{AdaIN}(f(c), f(s)) = \sigma(f(s)) \left(\frac{f(c) - \mu(f(c))}{\sigma(f(c))} \right) + \mu(f(s)), \quad (5)$$

applied independently per channel. This normalizes out the content image’s own channel-wise statistics (removing its native style) while leaving the *spatial pattern* of the content features untouched, then re-injects the style image’s channel-wise statistics onto that same spatial pattern. Where the Gram matrix captures $O(C_l^2)$ pairwise cross-channel correlations pooled over all spatial positions, AdaIN captures only $O(C_l)$ first- and second-order per-channel moments – a much smaller, closed-form, non-iterative statistic. No cross-channel correlation structure is modeled at all: each channel is treated independently.

3.2 Feed-forward architecture and training

Because AdaIN produces a target *feature tensor* $t = \text{AdaIN}(f(c), f(s))$ rather than a loss gradient with respect to pixels, a separate mechanism is needed to turn t into an actual image. Huang and Belongie train a decoder network g , mirroring the VGG encoder f up to relu4_1, whose sole job is to invert t back into plausible pixels: $g(t) \approx$ a stylized image. Because f (composed of ReLU and max-pooling operations) is not invertible, this decoder cannot be constructed analytically and must be learned from data, over a large corpus of (content, style) pairs drawn from MS-COCO and WikiArt, using:

The content loss enforces that the decoder’s output, once re-encoded, matches the AdaIN target t . Essentially, we want the spatial arrangement of information before and after decoding to be the same:

$$\mathcal{L}_c = \|f(g(t)) - t\|_2 \quad (6)$$

The style loss enforces that the channel-wise statistics of the output match those of the style image, aggregated across multiple layers $l \in \mathcal{L}$:

$$\mathcal{L}_s = \sum_{l \in \mathcal{L}} (\|\mu(\phi_l(g(t))) - \mu(\phi_l(s))\|_2 + \|\sigma(\phi_l(g(t))) - \sigma(\phi_l(s))\|_2) \quad (7)$$

Here, $\phi_l(g(t))$ denotes the activations coming from the encoder in a layer l when feeding $g(t)$ into the encoder. The two terms are combined into a single training objective, weighted by λ :

$$\mathcal{L} = \mathcal{L}_c + \lambda \mathcal{L}_s \quad (8)$$

with gradients updating **only the decoder’s weights**; the VGG encoder f remains frozen throughout, exactly as in Gatys et al. The decoder is trained *once*, over the full training distribution of content/style pairs, rather than re-solved per image pair.

At test time (i.e., for any new, unseen content/style pair), producing a stylized image requires exactly one encoder forward pass on c , one on s , one AdaIN computation, and one decoder forward pass – no iteration, no gradient descent on pixels, no per-image optimization loop at all.

4 Direct Comparison

4.1 Style representation: Gram matrix vs. channel statistics

The Gram matrix and AdaIN’s (μ, σ) pair are related but not equivalent: the diagonal of a Gram matrix is proportional to per-channel squared activation magnitude, closely related to σ^2 , but the off-diagonal entries – the cross-channel correlation structure that Gram-matrix loss explicitly penalizes – have no analogue in AdaIN at all. AdaIN is therefore a strictly coarser style descriptor. This is not incidental to its speed: it is precisely what makes AdaIN’s style representation *directly applicable to a feature map in closed form*, rather than only usable as a differentiable target within an iterative loss. A Gram matrix tells you how far your current image’s style is from the target; it does not, by itself, tell you how to edit a feature map to fix that discrepancy in one step. Matching a target Gram matrix in general requires iterative gradient-based search, because there is no closed-form feature edit that sets $G^l(p)$ to a target value while leaving spatial content undisturbed. Channel-wise mean/std, by contrast, is trivially and exactly settable in a single affine operation per channel – normalize out the old mean/std, inject the new ones. This algebraic difference is the root cause of the entire architectural divergence described next.

	Gatys et al.	AdaIN
Style statistic	Gram matrix $G^l \in \mathbb{R}^{C_l \times C_l}$	Channel mean/std $\mu, \sigma \in \mathbb{R}^{C_l}$
Order of statistic	Second-order, cross-channel	First- and second-order, per-channel
Captures cross-channel correlation	Yes	No
Computed at	Multiple layers, summed	Single layer for the transform (relu4_1); multiple layers used only for the auxiliary training loss
Cost per evaluation	$O(C_l^2 N_l)$	$O(C_l N_l)$
Closed-form transform available	No (only a loss term; no direct feature edit)	Yes (AdaIN directly rewrites features in closed form)

Table 1: Style representation: Gram matrix vs. channel-wise statistics.

	Gatys et al.	AdaIN
What is optimized	Pixels of the output image, per (content, style) pair	Decoder network weights, once, over a training set
Optimization variable	$p \in \mathbb{R}^{3 \times H \times W}$, re-initialized each run	Decoder parameters θ_g , fixed after training
Iterations at inference time	$\mathcal{O}(10^2\text{--}10^3)$ gradient steps	1 forward pass
Generalization to new pairs	None – new optimization required for every new pair	Immediate – any new (content, style) pair reuses the trained decoder
Typical inference time (GPU)	Seconds to minutes per image	Real time (fractions of a second)
Where VGG is used	Sole role: differentiable loss/gradient source for pixel optimization	Same role, plus provides the frozen feature space the decoder is trained to invert

Table 2: Iterative per-image optimization vs. amortized feed-forward inference.

4.2 From iterative optimization to feed-forward inference

Gatys et al. treat style transfer as a *search* problem: for each new pair of images, find the pixel array that jointly minimizes content and style loss, via backpropagation through a frozen VGG all the way to the pixels themselves. Nothing is learned across image pairs; each stylization is solved independently from scratch, which is what makes the method slow – the cost scales with the number of optimization iterations, each of which requires a full forward/backward pass through VGG on a full-resolution image.

AdaIN instead *amortizes* the cost of stylization: rather than solving an optimization problem per image pair, it trains a single decoder network, once, to approximately invert the AdaIN-transformed feature space for the entire distribution of content/style pairs seen during training (and, empirically, for unseen pairs too, since the transform generalizes arbitrarily at test time). All the expensive iterative search happens once, offline, during training; at inference, stylizing a new pair costs exactly one encoder pass, one closed-form statistic alignment, and one decoder pass – no gradient descent on pixels at test time at all.

4.3 Why the representation change enables the speed change

The causal relationship between the three points in this comparison is worth making explicit, since it is easy to treat them as three separate, coincidental differences between the papers:

1. Gatys et al.’s Gram-matrix style loss has **no closed-form inverse**: given a target Gram matrix, there is no direct formula for the feature map (or image) that produces it. The only way to find an image whose Gram matrix matches a target is iterative gradient-based search.
2. Because there is no closed-form way to *apply* a Gram-matrix-based style edit, style transfer under this representation is necessarily framed as a loss to be minimized by search, per image, at test time.
3. AdaIN’s channel-wise mean/std statistic **does** have a closed-form application: given target (μ, σ) , the exact feature map that has those statistics (while preserving the input’s normalized spatial pattern) is given directly by the AdaIN formula – no search required.
4. Because the style edit itself is now a single algebraic operation rather than an optimization target, the only remaining unsolved problem is turning an edited feature map back into pixels – and this sub-problem, unlike the whole stylization problem, is exactly the kind of fixed input-output mapping a feed-forward network can learn once and reuse indefinitely.

In short: the shift from Gram matrix to channel-wise statistics is not a parallel, independent improvement alongside the shift from iterative to feed-forward computation – it is the specific algebraic property (closed-form invertibility of the style operation) that makes the feed-forward shift possible in the first place. A Gram-matrix-based feed-forward method is not merely undiscovered by Gatys et al.; it is substantially harder to construct precisely because there is no single-step feature edit that realizes a Gram-matrix target, whereas AdaIN was designed around a statistic chosen specifically because it does admit one.

4.4 Quality and expressiveness trade-off

The comparison is not one-directional in favor of AdaIN. Because the Gram matrix retains cross-channel correlation structure that AdaIN discards, Gatys et al.’s method can, in principle, capture more complex style characteristics – certain correlated textures and stroke patterns that manifest as off-diagonal Gram matrix structure are simply invisible to a method that only matches per-channel means and variances. In practice, AdaIN’s stylizations are often visually strong but occasionally coarser or less textured than a fully converged Gatys optimization on the same pair, particularly for style images with strong cross-channel texture correlations. The comparison is therefore best framed as a trade-off between representational richness (Gram matrix, expensive, iterative) and representational efficiency (channel statistics, cheap, closed-form, feed-forward), rather than a strict improvement of one method over the other.

5 Conclusion

Gatys et al. and AdaIN share the same underlying empirical premise – that a frozen, ImageNet-pretrained VGG separates content (spatial activation pattern) from style (activation statistics) – but diverge in how finely they represent style and, as a direct consequence, in how they compute a stylized output. Gatys et al.’s Gram-matrix representation is expressive but algebraically unwieldy, forcing style transfer to be solved as a slow per-image optimization problem. AdaIN’s channel-wise mean/variance representation sacrifices cross-channel correlation structure in exchange for a closed-form, single-step style edit, which converts the remaining problem (feature-to-pixel inversion) into something a feed-forward network can learn once and apply to

arbitrary new content/style pairs in real time. The move from Gram matrix to channel statistics and the move from iterative optimization to feed-forward inference are thus not two separate contributions of the AdaIN paper, but one contribution with two visible consequences.